

System Programming Project 3

담당 교수 : 이영민

이름 : 이동협

학번 : 20211562

1. 개발 목표

이 프로젝트는 다수 클라이언트가 동시에 주식서버에 접속하여 현재 주식 정보를 조회하고, 주식을 사고팔 수 있는 서버-클라이언트 기반의 주식 거래 서버를 구현하는 것이다.

서버는 주식 데이터를 이진 탐색 트리 형태로 관리하고 이를 stock.txt의 텍스트 파일로 저장한다. 또한, 클라이언트의 요청에 따라 `show`, `buy`, `sell` 명령을 처리한다.

동시 접속을 지원하기 위해 서버는 event-driven 방식(select 기반), thread 기반으로 구성되며, 각 클라이언트의 요청을 처리한 후 변경된 주식 정보를 모든 클라이언트의 연결이 종료되면 파일에 저장한다.

2. 개발 범위 및 내용

A. 개발 범위

1. Task 1: Event-driven Approach

Event-driven 방식에서는 `select()` System Call을 활용하여 단일 스레드 내에서 여러 클라이언트의 연결을 동시에 감지하고 처리할 수 있도록 설계했다.

Stock Server는 클라이언트로부터의 연결 요청을 감지하고, 각 클라이언트의 요청이 들어올 때마다 해당 명령(show, buy, sell)을 처리(handle_command)하여 응답을 보낸다. 클라이언트의 연결이 종료되면 전체 활성 클라이언트 수를 관리하며, 모든 클라이언트가 종료된 시점에 주식 데이터를 `stock.txt`에 저장한다.

2. Task 2: Thread-based Approach

서버는 초기 실행 시 다수의 worker thread를 생성하고, 공유 작업 큐(sbuf)를 통해 클라이언트의 연결 요청을 각 스레드에 분배한다. 이를 통해 여러 클라이언트가 동시에 접속하여 주식 정보를 조회하고, 주식을 사고파는 명령(show, buy, sell)을 병렬적으로 처리할 수 있다.

주식 데이터는 이진 탐색 트리로 관리되며, command 처리 시 개별 노드 단위의 동기화를 통해 일관성을 보장한다. 클라이언트가 모두 종료되면 서버는 자동으로 주식 정보를 파일에 저장하며, 서버는 계속 실행 상태를 유지한다.

3. Task 3: Performance Evaluation

Multiclient.c를 활용하여 task1, task2의 수행시간을 비교하여 event-driven 서버와 thread-based 서버의 performance를 비교한다.

B. 개발 내용

- Task1 (Event-driven Approach with select())

- ✓ Multi-client 요청에 따른 I/O Multiplexing 설명

Event-driven 방식은 select() System Call 을 사용하여 단일 스레드로 다수의 클라이언트 소켓을 동시에 감시하는 구조이다.

서버는 listening socket 과 각 client socket 을 fd_set 에 등록하고, select()를 통해 어느 소켓에 데이터가 준비되었는지 비동기적으로 감지한다.

select()는 I/O Multiplexing 기술로, CPU 가 blocking 없이 여러 소켓의 상태를 동시에 확인할 수 있도록 해주며 각 클라이언트마다 별도의 스레드를 생성하지 않아도 다수의 클라이언트 요청을 처리할 수 있다.

- ✓ epoll과의 차이점 서술

- select()는 모든 파일 디스크립터를 매번 전부 읽어야($O(N)$)하기 때문에 클라이언트 수가 많아질수록 성능이 급격히 저하된다.
- 반면, epoll 은 리눅스 전용 고성능 I/O Multiplexing API 로, $O(1)$ 의 시간으로 이벤트 감지가 가능하며 Ready 상태의 fd 만 관리하므로 수천~수만 개의 소켓을 효율적으로 처리할 수 있다.
- 또한 select()는 FD 개수에 FD_SETSIZE 제한이 있지만, epoll 은 사실상 이 제한이 없고 비동기 이벤트 기반 구조로 더욱 확장성 있는 서버 구현이 가능하다.
- 따라서 소수 클라이언트 연결에서는 select()로도 충분하나, 다중 접속이 필요한 서버에서는 epoll 이 더 적합하다.

- Task2 (Thread-based Approach with pthread)

- ✓ Master Thread의 Connection 관리

이 서버에서 Master Thread는 클라이언트의 연결 요청을 수신하고, 이를 sbuf 에 저장한다.

main() 함수 내에서 Master Thread는 다음 작업을 반복적으로 수행한다:

1. Accept()를 통해 새로운 클라이언트 연결을 수락하고
2. 클라이언트의 connfd를 sbuf_insert()를 통해 subf에 추가
3. Worker Thread가 이 버퍼에서 작업을 가져가 처리하도록 한다.

Master Thread는 클라이언트의 실제 요청을 직접 처리하지 않고 연결만 수락하고 작업 큐에 전달한다.

✓ Worker Thread Pool 관리하는 부분에 대해 서술

서버는 NTHREADS 개의 Worker Thread를 생성하고 이 thread들은 thread() 함수에서 while loop을 돌며 다음을 반복 수행한다:

1. sbuf_remove()로 sbuf에서 클라이언트의 연결을 하나 꺼내고
2. 그 연결에 대해 handle_command()를 통해 command 처리
3. 클라이언트 연결이 종료되면 Close() 실행하고 active_clients 감소
4. 마지막 클라이언트가 종료되었을 경우 save_stock()를 호출하여 변경된 주식 정보를 파일에 저장

스레드 간 동기화를 위해 주식 노드에는 sem_t mutex가 사용되며, active_clients와 같은 공유 자원(정보 저장 타이밍을 위한 공유 자원)도 client_cnt_mutex를 통해 보호된다.

- Task3 (Performance Evaluation)

✓ 얻고자 하는 metric 정의, 그렇게 정한 이유, 측정 방법 서술

Metric은 총 3개를 정의하였고 다음과 같다.

Metric1- show/buy/sell이 모두 랜덤으로 동작하는 프로젝트에서 주어진 multiclient

Metric2- 명령 중 show만 입력하는 multiclient

Metric3- 명령 중 buy/sell(50% 확률)만 입력하는 multiclient

MAX_CLIENT 1000, ORDER_PER_CLIENT 10, STOCK_NUM 10, BUY_SELL_MAX 10

(위 값들은 모든 Metric에서 동일하게 사용했다.)

이렇게 3개의 Metric을 task1, task2의 서버에 대해 2, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000 client가 실행되었을 때의 elapsed time을 측정하였다. Gettimeofday(&start, 0)을 클라이언트 수만큼 반복하는 while loop 이전에 실행하고 이 while loop이 끝났을 때 gettimeofday(&end, 0)을 실행하여 이 사이의 간격을 측정했다.

우선, Metric1을 이렇게 정한 이유는 그냥 일반적인 주식 서버처럼 정보 출력 요청과 정보 조작(buy/sell)이 제한 없이 이루어질 때 event-driven 서버와 thread 서버가 각각 어떤 performance를 보이는지 알아보기 위함이었다.

또한, Metric2와 3을 정한 이유는 Metric1이 random하게 show/buy/sell을 요청하기 때문에 일관된 performance를 확인하기 위해 같은 종류의 명령만 요청하는 metric2와 3을 추가하였다.

✓ Configuration 변화에 따른 예상 결과 서술

Client 수가 많아지면서 event, thread 기반 서버 모두 실행 시간이 길어지나 task 별로 차이가 존재할 것이다. event 서버는 단일 thread 로 클라이언트들의 명령 요청을 관리하고 thread 서버는 다수의 worker thread 가 각자 클라이언트 명령 요청을 관리하기 때문에 Thread based 서버가 더 적은 수행 시간과 더 많은 시간당 처리량을 보여줄 것으로 기대한다.

C. 개발 방법

Task 공통 구현 부분

주식 정보를 저장하는 bst를 구현하기 위해 주식 번호, 남은 개수, 가격, readcnt, mutex, left child node, right child node를 멤버 변수로 가지는 tree_node를 구현했고 이를 stock.txt에서 불러오고 명령 처리를 할 때 값을 바꾸고 마지막으로 다시 stock.txt에 저장하는 함수들을 load_stock, insert_stock, save_stock_recursive, save_stock, find_stock에 구현하였다. 또한, show명령 처리를 할 때 bst를 inorder traverse하기 위한 inorder 함수도 구현하였다.

또한, sigint signal이 들어왔을 때 서버가 종료될 수 있도록 sigint_handler, multiclident.c가 서버의 응답을 받을 수 있도록 응답(Rio_writen)해주는 padded_write를 구현했다.

마지막으로, 연결됐던 모든 client의 연결이 종료됐을 때 stock.txt에 주식 정보를

저장하기 위해 active_clients 변수를 선언하고 semaphore(client_cnt_mutex)를 통해 클라이언트가 모두 종료되면 save_stock을 호출해 stock.txt에 저장한다.

각 명령을 처리하는 handle_command 함수는 show가 들어오면 inorder를 통해 버퍼(bigbuf)에 현재 bst를 넣어준다. 이후 padded_write를 통해 client에게 응답을 전송한다. Buy/sell 명령의 경우 find_stock을 통해 해당 id를 가진 node를 찾고 해당 노드의 mutex의 semaphore를 획득하고, 남은 주식 수를 변경해주고 padded_write를 통해 buy/sell 성공 여부를 알려준다. 이후 semaphore를 해제한다.

Task1 구현 부분

main함수에서 listenfd를 통해 클라이언트의 연결 요청을 감지하고, clientfd를 통해 각 클라이언트에서 오는 데이터를 감지하고 select()를 통해 어떤 fd가 ready 상태가 되었는지 감지하고 이 ready된 socket에 대해 read 혹은 accept 처리를 한다.

Task2 구현 부분

서버는 main()에서 NTHREADS 개의 worker thread를 미리 생성하여 thread pool을 구성한다. 그리고 클라이언트의 연결은 Accept()를 통해 수락한 후 공유 버퍼(sbuf)에 저장한다.

Sbuf_t 구조체는 버퍼, 버퍼 크기, front rear 인덱스, semaphore인 mutex, slots, items로 구성하였다.

이 공유 버퍼를 초기화하는 sbuf_init, 버퍼에 정보를 추가하는 sbuf_insert, 제거하는 sbuf_remove를 구현했다.

3. 구현 결과

select를 이용한 event-driven 방식의 서버는 정상작동했고, 다수 클라이언트가 동시에 접속하여 (show/buy/sell) 명령을 수행하는 데 문제가 없었다.

클라이언트 연결 종료 시 서버가 자동으로 주식 정보를 stock.txt 파일에 저장하는 기능도 정상적으로 동작했다.

Thread 기반 서버 또한 정상작동했다. 다수의 클라이언트 연결이 동시에 처리되며, 각 명령 처리 show/buy/sell과 동기화 기능, sbuf에 대한 세마포어 동기화도 적절히 작동하여 data race없이 안정적으로 실행됐다.

4. 성능 평가 결과 (Task 3)

- 공통 조건

모든 Metric에서 아래와 같은 define 값을 사용했다.

MAX_CLIENT 1000, ORDER_PER_CLIENT 10, STOCK_NUM 10, BUY_SELL_MAX 10

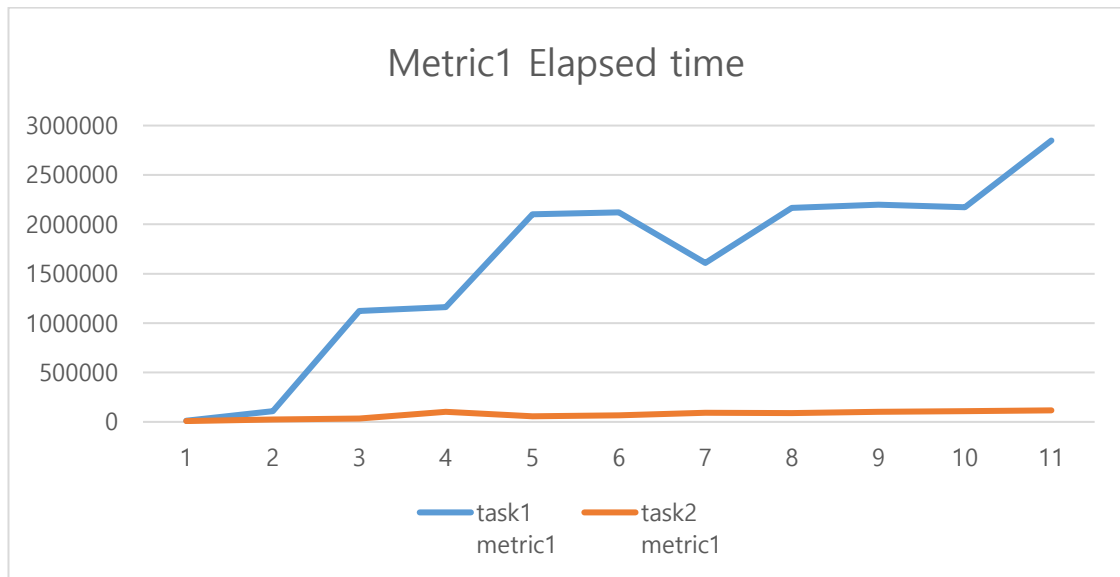
또한 usleep 시간을 1000000으로 고정했다.

Elapsed time은 gettimeofday 차이 값에서 1 * ORDER_PER_CLIENT 만큼(이 실험에서는 10초로 고정) 빼주어 stockserver에서 순수하게 지연된 시간(multiclient에서 나머지 동작을 수행하는 미세한 시간은 모두 일정하니 무시)을 기준으로 했다.

- Metric 1

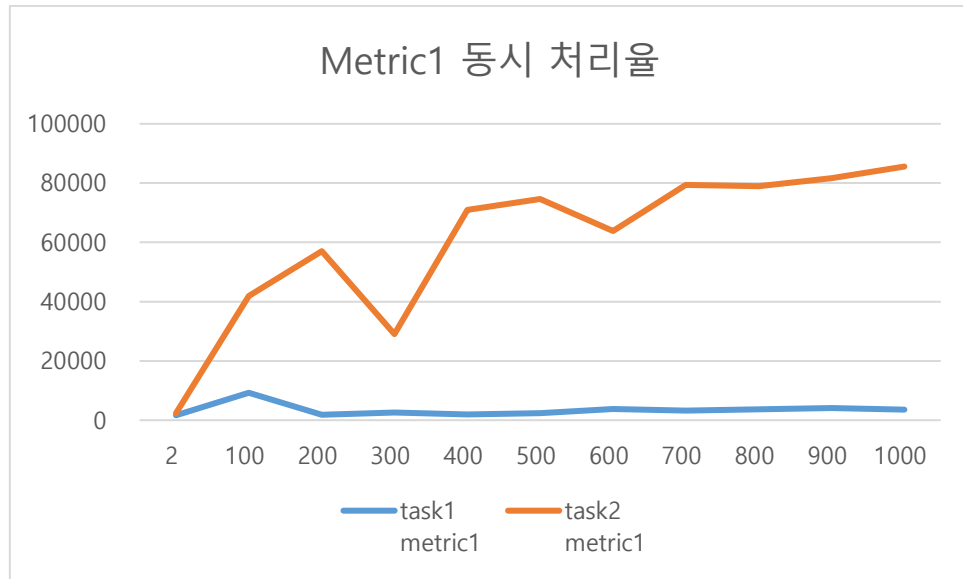
Elapsed Time(단위 microsecond)

clientnum	2	100	200	300	400	500	600	700	800	900	1000
task1 metric1	12130	108107	1121962	1161108	2101489	2121801	1608641	2167592	2197996	2172450	2848460
task2 metric1	8442	23843	35043	103406	56335	66986	94015	88222	101285	110250	116856



동시 처리율(처리량/초)

clientnum	2	100	200	300	400	500	600	700	800	900	1000
task1 metric1	1649	9250	1783	2584	1903	2356	3730	3229	3640	4143	3511
task2 metric1	2369	41941	57073	29012	71004	74642	63820	79345	78985	81633	85575

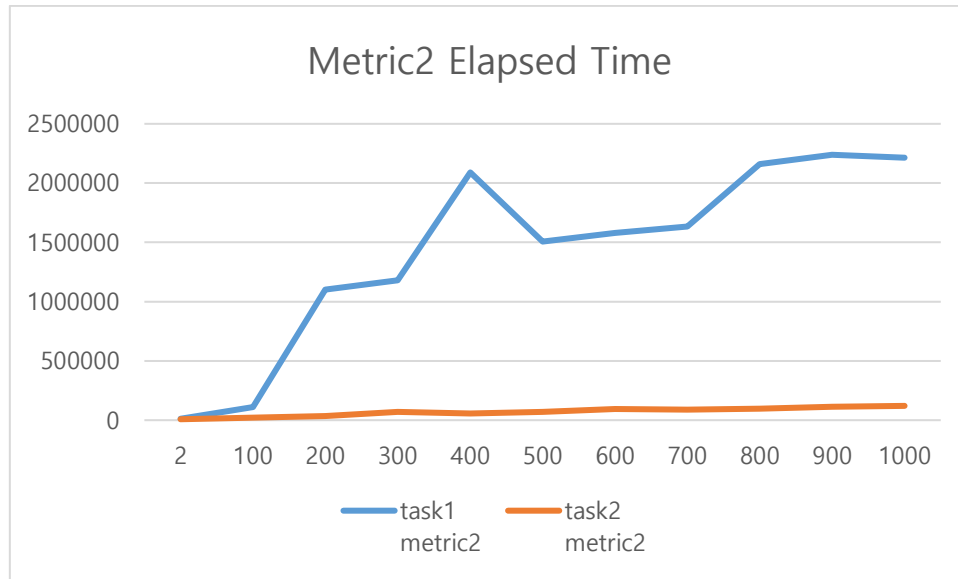


Show/buy/sell 명령이 혼재되어있어 일정한 처리율을 보이진 못했지만 전반적으로 thread 기반 서버가 event 기반 서버보다 처리율이 훨씬 높았다. 또한 동시처리율 그래프의 기울기(처리량 증가에 따른 동시처리율 증가 폭)이 event 서버는 거의 일정했으나 thread서버는 처음에 가파르다가 client수가 증가하자 완만해지는 것을 확인할 수 있었다.

- Metric 2

Elapsed Time(단위 microsecond)

clientnum	2	100	200	300	400	500	600	700	800	900	1000
task1 metric2	12196	110504	1102843	1180483	2090953	1507816	1580236	1633307	2159822	2239051	2212931
task2 metric2	7773	22452	35661	69809	56077	68709	93775	88633	98268	113087	120676



동시 처리율(처리량/초)

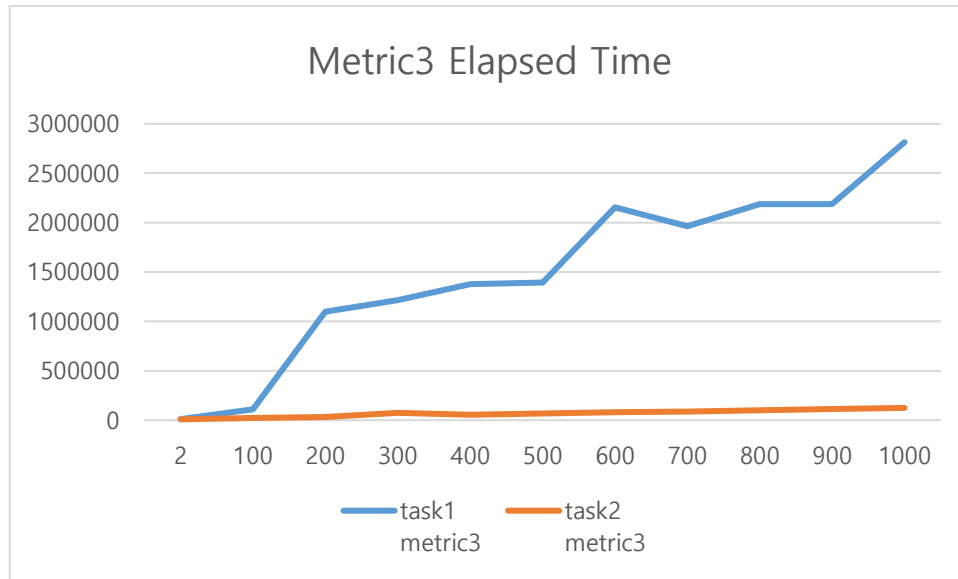
clientnum	2	100	200	300	400	500	600	700	800	900	1000
task1 metric2	1640	9049	1813	2541	1913	3316	3797	4286	3704	4020	4519
task2 metric2	2573	44539	56084	42974	71330	72771	63983	78977	81410	79585	82867

Metric2 또한 전반적으로 thread 기반 서버가 event 기반 서버보다 처리율이 훨씬 높았다. 또한 동시처리율 그래프의 기울기(처리량 증가에 따른 동시처리율 증가 폭)이 event 서버는 거의 일정했으나 thread서버는 처음에 가파르다가 client수가 증가하자 완만해지는 것을 확인할 수 있었다.

- Metric 3

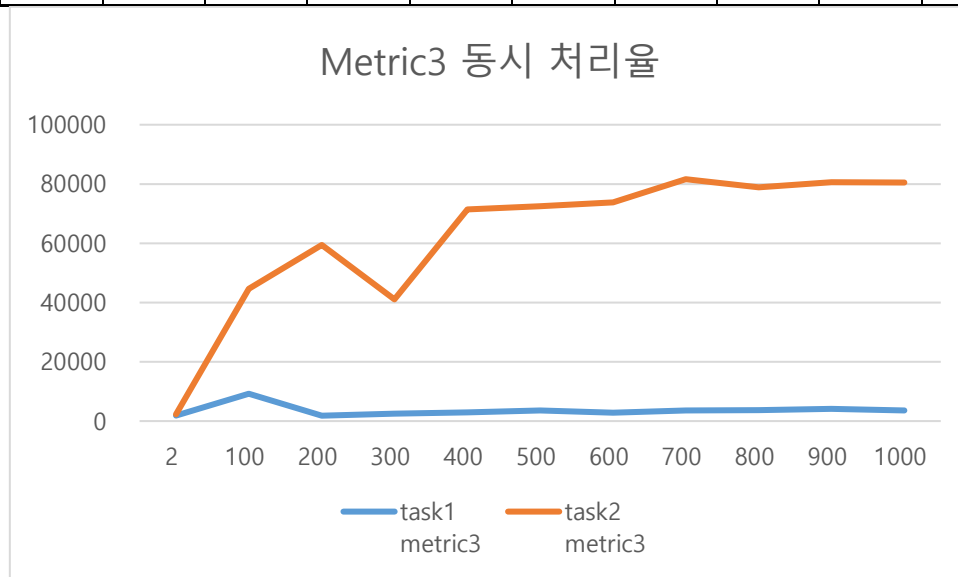
Elapsed Time(단위 microsecond)

clientnum	2	100	200	300	400	500	600	700	800	900	1000
task1 metric3	10826	108338	1097597	1213459	1375447	1392936	2154834	1962245	2186525	2188135	2814130
task2 metric3	8785	22425	33652	73065	56010	68961	81272	85765	101447	111726	124153



동시 처리율(처리량/초)

clientnum	2	100	200	300	400	500	600	700	800	900	1000
task1 metric3	1847	9230	1822	2472	2908	3590	2784	3567	3659	4113	3553
task2 metric3	2277	44593	59432	41059	71416	72505	73826	81618	78859	80554	80546



Metric3 또한 전반적으로 thread 기반 서버가 event 기반 서버보다 처리율이 훨씬 높았다. 또한 동시처리율 그래프의 기울기(처리량 증가에 따른 동시처리율 증가 폭)이 event 서버는 거의 일정했으나 thread서버는 처음에 가파르다가 client수가 증가하자 완만해지는 것을 확인할 수 있었다.

- 전반적인 분석

우선, Metric2(show만) Metric3(buy/sell만)을 비교했을 때 그렇게 큰 차이가 있지는 않지만 대체적으로 metric3가 더 큰 지연시간을 가졌던 것으로 보아 buy/sell 작업에서 공유 자원을 변경하기에 semaphore를 사용하는 과정이 살짝의 지연시간을 더한 것으로 볼 수 있다.

그렇다면 왜 thread가 event보다 훨씬 높은 동시 처리율을 보였을까. 먼저, select는 클라이언트가 증가함에 따라 모든 fd를 매번 검사하기 때문에 이는 상당한 overhead를 가져온다. 또한 하나의 loop에서 모든 I/O를 순차적으로 처리했기에 한 process만 block되도 전체가 지연된다.

반면, thread 서버에서는 각 연결을 thread가 병렬로 처리하기에 클라이언트가 증가해도 동시에 N thread가 병렬적으로 응답할 수 있다. 따라서 단일 스레드로 처리하는 event-driven 기반 서버보다 thread 기반 서버가 훨씬 빨랐던 것이다.

각 elapsed time 캡처

Task1 metric1

```
elapsed time: 10012130 microseconds
```

```
elapsed time: 10108107 microseconds
```

```
elapsed time: 11121962 microseconds
```

```
elapsed time: 12101489 microseconds
```

```
elapsed time: 11161108 microseconds
```

```
elapsed time: 12121801 microseconds
```

```
elapsed time: 11608641 microseconds
```

```
elapsed time: 12167592 microseconds
```

```
elapsed time: 12197996 microseconds
```

```
elapsed time: 12172450 microseconds
```

```
elapsed time: 12167592 microseconds
```

Task1 metric2

elapsed time: 10012196 microseconds

elapsed time: 10110504 microseconds

elapsed time: 11102843 microseconds

elapsed time: 11180483 microseconds

elapsed time: 12090953 microseconds

elapsed time: 11507816 microseconds

elapsed time: 11580236 microseconds

elapsed time: 11633307 microseconds

elapsed time: 12159822 microseconds

elapsed time: 12239051 microseconds

elapsed time: 12212931 microseconds

Task1 metric3

elapsed time: 10010826 microseconds

elapsed time: 10108338 microseconds

elapsed time: 11097597 microseconds

elapsed time: 11213459 microseconds

elapsed time: 11375447 microseconds

elapsed time: 11392936 microseconds

elapsed time: 12154834 microseconds

elapsed time: 11962245 microseconds

elapsed time: 12186525 microseconds

elapsed time: 12188135 microseconds

elapsed time: 12814130 microseconds

Task2 metric1

elapsed time: 10008442 microseconds

elapsed time: 10018527 microseconds

elapsed time: 10035043 microseconds

elapsed time: 10103406 microseconds

elapsed time: 10056335 microseconds

elapsed time: 10066986 microseconds

elapsed time: 10094015 microseconds

elapsed time: 10088222 microseconds

elapsed time: 10101285 microseconds

elapsed time: 10110250 microseconds

elapsed time: 10116856 microseconds

Task2 metric2

elapsed time: 10007773 microseconds

elapsed time: 10022452 microseconds

elapsed time: 10035661 microseconds

elapsed time: 10069809 microseconds

elapsed time: 10056077 microseconds

elapsed time: 10068709 microseconds

elapsed time: 10093775 microseconds

elapsed time: 10088633 microseconds

elapsed time: 10098268 microseconds

elapsed time: 10113087 microseconds

elapsed time: 10120676 microseconds

Task2 metric3

elapsed time: 10111726 microseconds

elapsed time: 10101447 microseconds

elapsed time: 10085765 microseconds

elapsed time: 10081272 microseconds

elapsed time: 10068961 microseconds

elapsed time: 10056010 microseconds

elapsed time: 10073065 microseconds

elapsed time: 10033652 microseconds

elapsed time: 10022425 microseconds

elapsed time: 10008785 microseconds